Fork Bomb для Flutter

Написано Филиппом Никифоровым8 сентября 2022 г.

Приложения Flutter можно найти в проектах по анализу безопасности или программах bugbounty. Чаще всего такие активы просто игнорируются из-за отсутствия методологий и способов их обратного проектирования. Я решил больше не пропускать это и разработал инструмент **reFlutter** . В этой статье описываются результаты моих исследований.

Краткое содержание

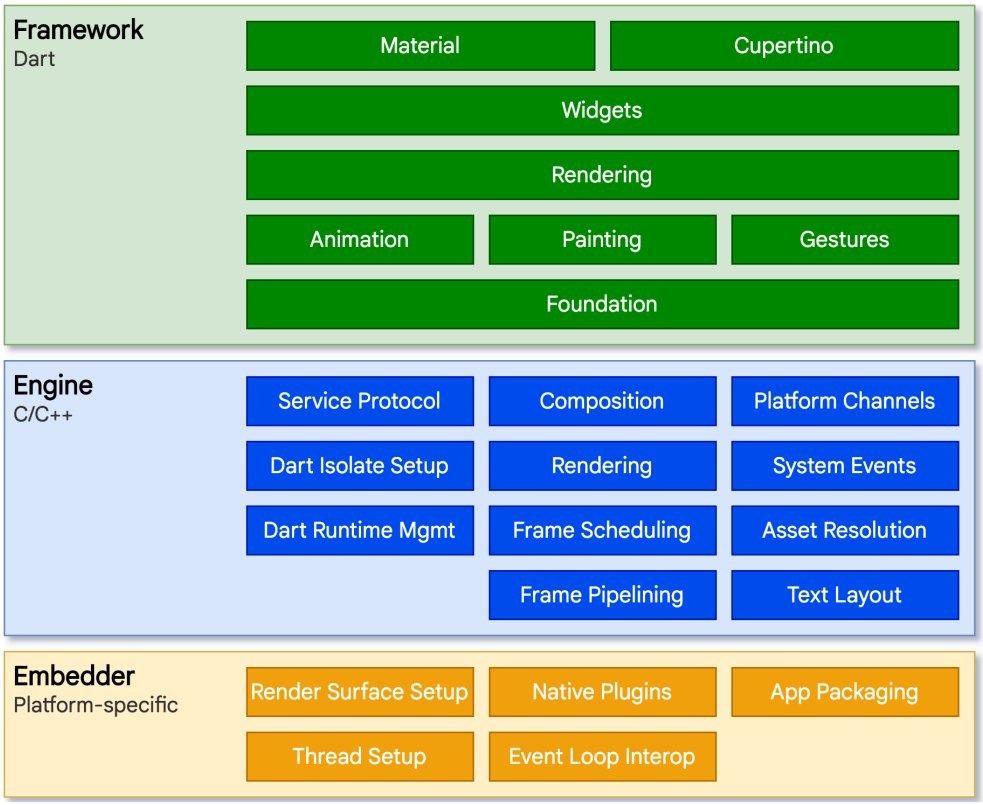
Отчет начинается с краткого обзора Flutter SDK, за которым следует рассмотрение компиляции простого мобильного приложения. Затем я покажу вам, как собрать Flutter самостоятельно, как он построен на CI/CD от Google, какие типы сборок существуют и чем версии отличаются друг от друга. Мы:

- Расскажите о конкретном подходе к обратному проектированию Flutter
  - Написать утилиту
  - Анализ исправлений для исходного кода DartVM
  - Создать Docker-контейнер
- Демонстрация на примере приложения BMW, перехват трафика в BurpSuite и захват аргументов функций через Frida
- Перекомпилируйте Engine вручную с помощью Docker
  - Выясните, как найти и сопоставить правильный коммит
  - Создание патчей для dev-сборки
  - Применить этот движок к приложению

Обзор архитектуры

Flutter — это SDK с открытым исходным кодом от Google для разработки кроссплатформенных приложений. Его цель — поставлять приложения, которые выглядят естественно на разных платформах, допуская различия в поведении прокрутки и типографике. Flutter создан на C, C++, Dart и Skia.

Flutter состоит из трех архитектурных слоев, но в контексте этой статьи мы рассмотрим только Engine и Framework.



Framework — это кроссплатформенный слой, написанный на языке Dart. Он включает в себя богатый набор платформ, макетов и базовых библиотек. Многие высокоуровневые функции, которые могут использовать разработчики, реализованы в виде пакетов, включая плагины платформы, такие как camera , webview , и другие функции, такие как http и animation .

Engine — это переносимая среда выполнения для хостинга приложений Flutter, которая содержит требуемый SDK для Android, iOS или Windows; в основном она написана на C++ и предоставляет примитивы для поддержки всех приложений Flutter. Движок включает пакет dart-sdk , который обеспечивает низкоуровневую реализацию: файловый и сетевой ввод-вывод, а также Dart VM и цепочку инструментов компилятора.

Разработчики приложений Flutter пишут код на языке Dart с использованием Framework. Этот код

выполняется в Dart VM, которую предоставляет Engine. При создании приложения для определенной платформы будет использоваться соответствующий Engine, скомпилированный специально для нее. Благодаря этой архитектуре, где можно менять платформу для уже существующего кода, Flutter является кроссплатформенным.

To compile a Flutter application, Engine is used to create an AOT AppSnapshot containing precompiled machine code: the Framework source code and the developers' source code. This article focuses on AOT, because this is the Snapshot type used in release builds.

Let's use the standard Flutter project for Android (in which all necessary libraries have been pre-placed for convenience) to see how the application is compiled.

Go to the android folder, run the build:

```
~/flutter_app/android$ ./gradlew -
Pverbose=true -Ptarget-platform=android-
arm64 -Ptarget=lib/main.dart
assembleRelease
```

Arguments:

`-Ptarget-platform` – select the architecture we need [android-x64, android-arm, android-arm64]
`-Ptarget` – path to the file with the main function of the application

The build process is underway:

```
> Task :app:compileFlutterBuildRelease
[  +11 ms] dart-sdk/bin/dart
artifacts/engine/linux-
x64/frontend_server.dart.snapshot --sdk-
root
artifacts/engine/common/flutter_patched_sd
k_product/ --target=flutter -
Ddart.developer.causal_async_stacks=false
-Ddart.vm.profile=false -
Ddart.vm.product=true --bytecode-
options=source-positions --aot --tfa --
packages .packages --output-dill app.dill
package:flutter_app/main.dart
[+7108 ms] kernel_snapshot: Complete
[   +3 ms] executing:
artifacts/engine/android-arm64-
release/linux-x64/gen_snapshot --
deterministic --snapshot_kind=app-aot-elf
--elf=libapp.so --strip --no-causal-async-
stacks --lazy-async-stacks app.dill
[+3668 ms] android_aot_release_android-
arm64: Complete
……….
[   +4 ms] build succeeded.
[   +6 ms] "flutter assemble" took
12,261ms.
```

Let’s look at the first executed command:

```
dart-sdk/bin/dart artifacts/engine/linux-
x64/frontend_server.dart.snapshot
--sdk-root
artifacts/engine/common/flutter_patched_sd
k_product
--target=flutter
-Ddart.developer.causal_async_stacks=false
-Ddart.vm.profile=false
-Ddart.vm.product=true
--bytecode-options=source-positions
--aot --tfa
--packages .packages
--output-dill app.dill
package:flutter_app/main.dart
```

This command starts the application `frontend_server.dart.snapshot` (CFE), written in Dart as part of the Engine. It compiles the Dart source code into an AST representation and saves it to a Dart Kernel Binary (.dill) file. You can find a description of this format here: https://github.com/dart-lang/sdk/wiki/Kernel-Documentation.

Arguments:

`--packages` – .packages file for compilation; has the format packageName:packageUri
`--output-dill` – output path for the generated .dill file
`--target` – target model that determines what core libraries are available [vm (default), flutter, flutter_runner, dart_runner, dartdevc]
`--tfa` – enable global type flow analysis and related transformations in AOT mode.
`--aot` – run compiler in AOT mode (enables whole-program transformations)

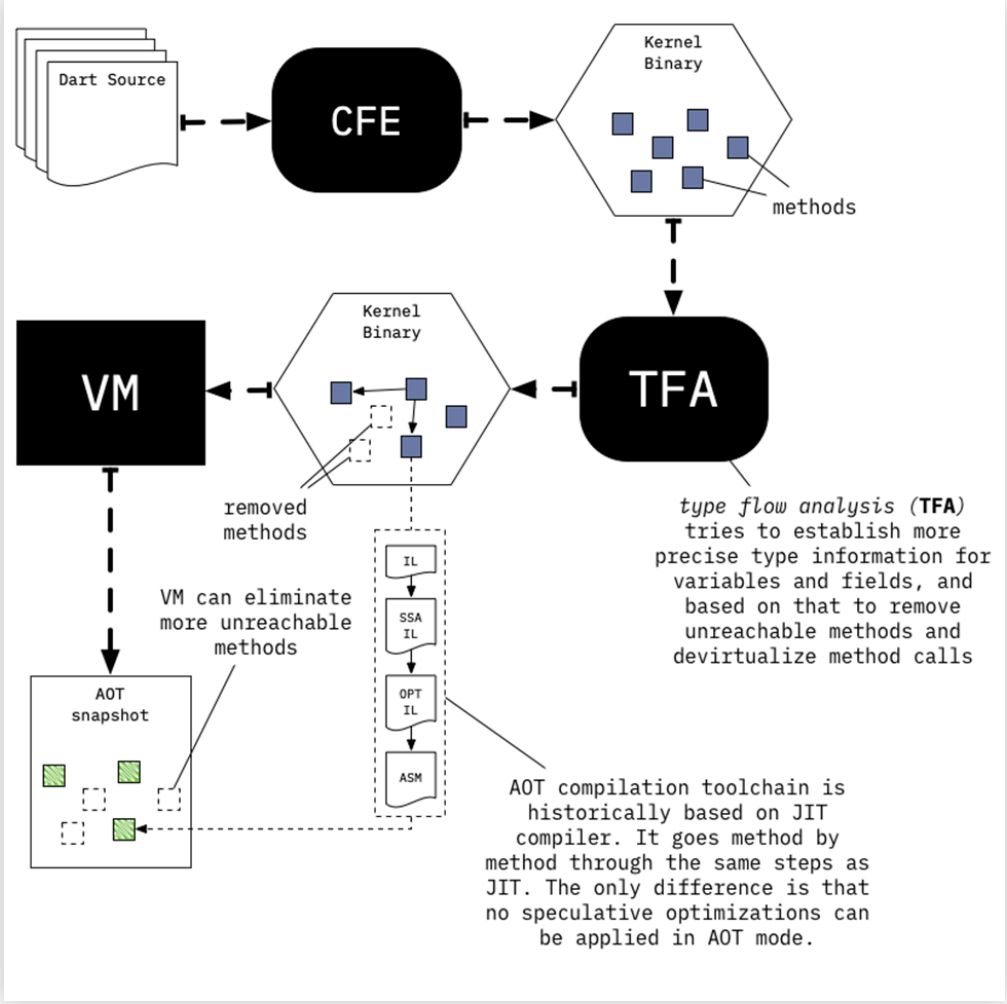
`package:flutter_app/main.dart` – path to the main function of the application

After saving the app.dill file, the second command is run.

```
artifacts/engine/android-arm64-release/linux-x64/gen_snapshot --deterministic --snapshot_kind=app-aot-elf --elf=libapp.so --strip --no-causal-async-stacks --lazy-async-stacks app.dill
```

Here the obtained dill file is specified and passed as an argument to gen_snapshot, which, when executed, generates an optimized FlowGraph (TFA) for the Dart code, before converting it to AOT binary machine code by writing it to a libapp.so file.

This is approximately how the compilation process looks in AOT:



Next, the obtained libapp.so is combined with resources, dex files, and the libflutter.so library into a single zip archive, which is signed and made ready-to-use release.apk

Structure of the release.apk file with comments:

```
├── AndroidManifest.xml
├── assets
│   └── flutter_assets
│       └── AssetManifest.json
├── classes.dex ──── //  Java (Dalvik
Executable)
├── kotlin ──── //  kotlin Metadata
├── lib
│   └── arm64-v8a
```

```
│       ├── libapp.so ──── //  Dart code
(App AOT Snapshot)
│       └── libflutter.so ──── //  Flutter
Engine (stripped version of Dart VM)
├── META-INF
├── res
└── resources.arsc
```

Since the target platform is android-arm64, the lib folder contains only one architecture, arm64-v8a.

The libflutter.so file (part of the Flutter Engine) contains the required functionality for using the OS (network, file system, etc.) and a stripped version of the DartVM. This version is known as precompiled runtime, which does not contain any compiler components and is incapable of loading Dart source code dynamically. However, it handles reading of sections, deserializing, and loading instructions (binary machine code) into executable memory from the ELF file libapp.so.

```
~$ readelf -Ws libapp.so

Symbol table '.dynsym' contains 6 entries:
   Num:    Value          Size Type
Bind   Vis                          Ndx
Name
     0: 0000000000000000     0 NOTYPE
LOCAL  DEFAULT  UND
     1: 0000000000001000     8 FUNC
GLOBAL DEFAULT    1 _kDartBSSData
     2: 0000000000002000 17792 FUNC
GLOBAL DEFAULT    2
_kDartVmSnapshotInstructions
```

```
     3: 0000000000007000 0x1f89c0 FUNC
GLOBAL DEFAULT    3
_kDartIsolateSnapshotInstructions
     4: 0000000000200000 32288 FUNC
GLOBAL DEFAULT    4 _kDartVmSnapshotData
     5: 0000000000208000 0x18c180 FUNC
GLOBAL DEFAULT    5
_kDartIsolateSnapshotData
```

Here we see the .text segments: `_kDartVmSnapshotInstructions`, `_kDartIsolateSnapshotInstructions`, and .rodata: `_kDartVmSnapshotData`, `_kDartIsolateSnapshotData`.

Dart has the Isolate abstraction — a structure with its own memory (heap) and usually with its own thread of control (mutator thread). All Dart code runs in an isolate. Multiple isolates can execute Dart code concurrently but cannot share any state directly and can only communicate by passing messages.

Let's have a look at the structure of the libapp.so file:

Isolate Instructions — `_kDartIsolateSnapshotInstructions` — Contains the AOT code that is executed by the Dart isolate. It must live in the **text** segment.

Isolate Snapshot — `_kDartIsolateSnapshotData` — Represents the initial state of the Dart heap and includes isolate specific information. Along with the VM snapshot, it helps in faster launches of the specific isolate. Should live in the **data** segment.

Dart VM Instructions — `_kDartVmSnapshotInstructions` — Contains AOT instructions for common routines shared between all Dart isolates in the VM. This snapshot is typically extremely small and mostly contains stubs. It must live in the **text** segment.

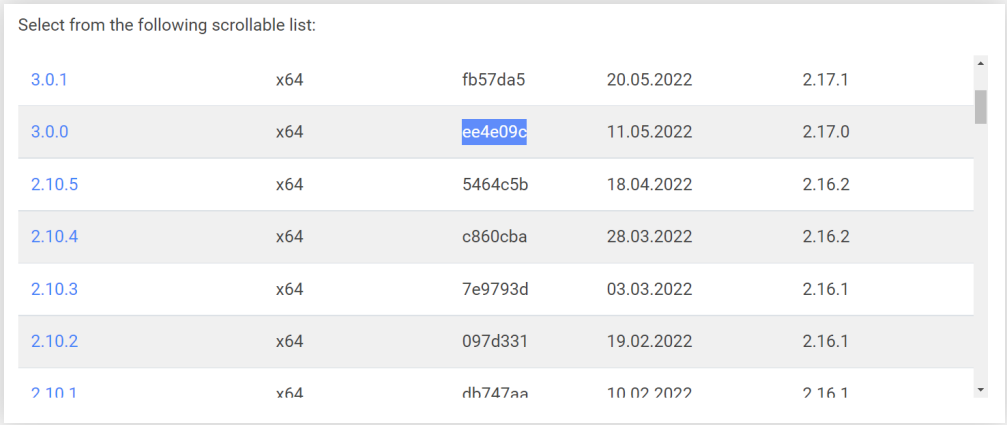
Dart VM Snapshot — `_kDartVmSnapshotData` — Represents the initial state of the Dart heap shared between isolates. Helps launch Dart isolates faster. Does not contain any isolate specific information. Mostly predefined Dart strings used by the VM. Should live in the **data** segment. From the VM's perspective, this needs to be loaded in memory with READ permissions and does not need WRITE or EXECUTE permissions. In practice, this means it should end up in .rodata when putting the snapshot in a shared library.

Each Engine (libflutter.so) stores an md5 hash (Snapshot_hash) to separate the build versions. This hash is generated on the basis of major changes in the Engine source code at compile time using the make_version.py script.

To check the compatibility of libflutter.so and libapp.so, the same Snapshot_hash is stored in them. If libflutter.so detects an invalid hash in libapp.so, the process terminates with an incompatibility error.

Flutter, like dart-sdk, is constantly under development: changes are made from version to version, performance is improved and language features are added. Therefore, when creating an application, the developer should consider which version of Flutter to use. There are three release versions in total: stable, beta and dev.

These versions (excluding dev) can be found here: docs.flutter.dev/development/tools/sdk/releases; the table lists: the Flutter version, the commit linked to this version (the Ref field), and the Dart version.



The commit makes it easy to find out which Flutter Engine version is included in the release; just substitute it here: github.com/flutter/flutter/blob/ee4e09c/bin/internal/engine.version.

Now we can study the particular Engine github.com/flutter/engine/blob/[engine.version]/DEPS. As you can see, the file DEPS contains dependencies, plus a dart-sdk commit, which is also easy to switch to github.com/dart-lang/sdk/blob/[dart_revision]/DEPS.

Each new version is developed according to the following steps (using github.com/flutter/engine as an example):

1. Development in progress.
2. A new-made build goes to **dev**.
3. At the beginning of the month, usually, the first Monday, when many changes happen, a build goes to **beta**.
4. When issues are tested and resolved, typically quarterly, a build goes to **stable**.

If the Engine source code is modified significantly, compilation will produce a different Snapshot_hash. Therefore, a lot of hashes will be generated for the dev version. But in the beta version, for instance, there are fewer changes than in the dev version, plus far fewer major edits, so there can be only one Snapshot_hash for beta, which may even match the hash of the stable version.

All Engine files (such as gen_snapshot, frontend_server.dart.snapshot, libflutter.so, dart-sdk) are uploaded here storage.googleapis.com after compilation:

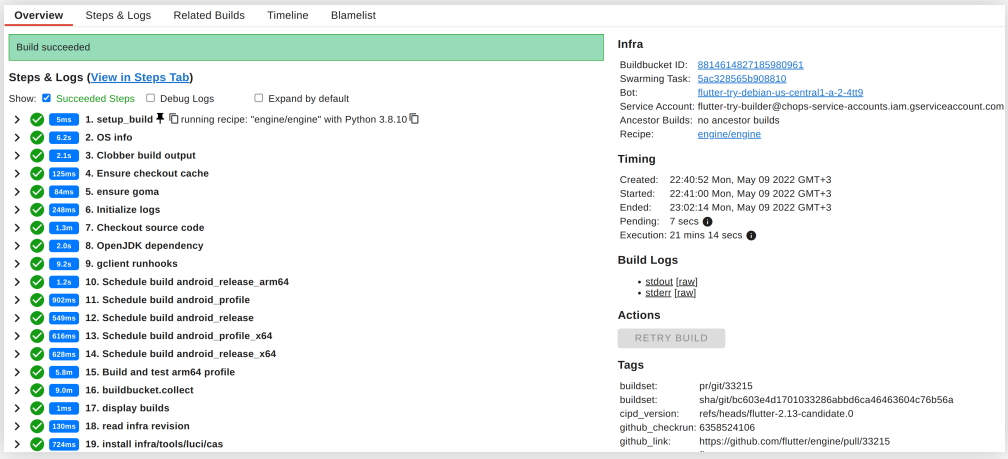
```
    {
      "hash":
"8f89f6505b941329a864fef1527243a72800bf4d"
,
      "channel": "beta",
      "version": "1.25.0-8.1.pre",
      "release_date": "2020-12-
16T21:55:19.340490Z",
      "archive":
"beta/linux/flutter_linux_1.25.0-8.1.pre-
beta.tar.xz",
      "sha256":
"8db28a4ec4dbd0e06c2c29e52560c8d9c7b0de8a9
4102c33764ec137ecd12e07"
    },
```

We use the hash (Flutter_Commit) to get the following links:

- ../flutter/92ae191c17a53144bf4d62f3863c110be08e3fd3/linux-x64/artifacts.zip
- ../flutter/92ae191c17a53144bf4d62f3863c110be08e3fd3/flutter_patched_sdk_product.zip
- ../flutter/92ae191c17a53144bf4d62f3863c110be08e3fd3/android-arm64-release/linux-x64.zip
- ../flutter/92ae191c17a53144bf4d62f3863c110be08e3fd3/dart-sdk-linux-x64.zip

These files can be downloaded and used to compile the Flutter project on Android, which we explored with you before. This is basically the whole part of the Engine needed to generate libapp.so.

Let's take a look at how the Flutter Engine is compiled. There is a description in the wiki: Setting-up-the-Engine-development-environment and Compiling-the-engine. To get more details, we can look at the repository and find Pull Request, where the flutter-dashboard element takes us to Google's CI/CD https://ci.chromium.org/ui/p/flutter/builders/try/Linux Android AOT Engine/35622/overview; this just so happens to upload artifacts after compilation to storage.googleapis.com.



Essentially the build goes as follows:

We install the necessary packages.

```
sudo apt-get install -y git wget curl
software-properties-common unzip python-
pip python lsb-release sudo apt-transport-
https
```

We clone the repository containing the required tools. Such as the Ninja build system and the gclient dependency management tool.

```
git clone
https://chromium.googlesource.com/chromium
/tools/depot_tools.git
```

We clone the Flutter Engine repository.

```
git clone
https://github.com/flutter/engine.git
```

We specify the directory for using depot_tools.

```
```

```
ROOT_DIR=`pwd`
export PATH=$PATH:$ROOT_DIR/depot_tools
```

We create a directory for using gclient.

```
mkdir customEngine
```

We create a .gclient configuration file, where `url` is the path to the Flutter Engine folder, and `deps_file` is the file's name with dependencies to be processed by gclient.

```
cd customEngine
echo 'solutions = [{"managed":
False,"name": "src/flutter","url":
"'$ROOT_DIR/engine'","custom_deps":
{},"deps_file": "DEPS","safesync_url":
"",},]' > .gclient
```

We start gclient, after which the Flutter Engine appears in the customEngine folder with all necessary dependencies (dart-sdk, skia).

```
gclient sync
```

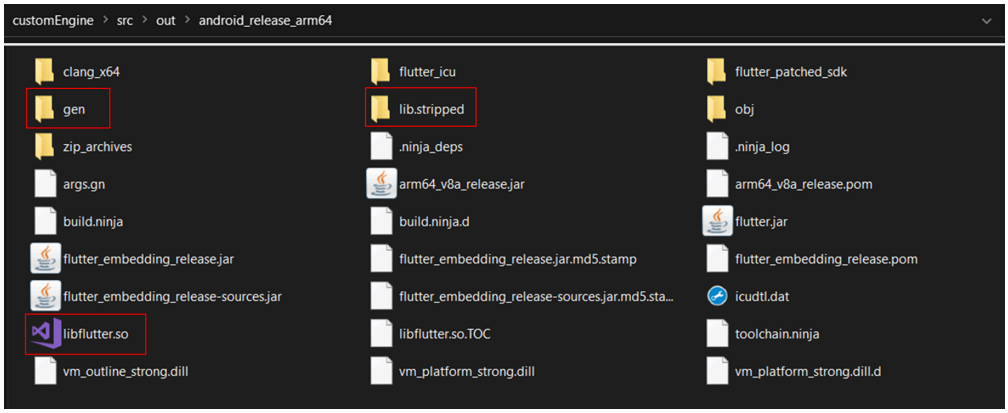
We install dependencies (ndk) to compile the Engine under Android.

```
sudo src/build/install-build-deps-android.sh --no-prompt
```

We use the GN and Ninja build systems, and select the desired architecture and release mode; at the output, we get the compiled Flutter Engine in the `src/out/android_release_arm64` folder.

```
src/flutter/tools/gn --android --android-cpu=arm64 --runtime-mode=release ninja -C src/out/android_release_arm64
```

This folder contains the files needed to create Snapshot: artifacts, flutter_patched_sdk_product, gen_snapshot. The lib.stripped folder contains the libflutter.so engine, which will be copied into one folder with libapp.so during the compilation of the APK.



Recompilation as a reverse engineering approach

We now know that the application code is stored in the libapp.so file, which the Engine reads. And the actual instructions for the code are in the `_kDartIsolateSnapshotInstructions` segment.

Unfortunately, the file is divided into segments with different types of objects and has a complex structure that requires deserialization; this is confusing and hinders the reverse engineering process, making it harder to know where the required instruction begins and what function it handles.

To understand how deserialization works we can learn the source code of libflutter.so, which is available here: github.com/dart-lang/sdk. Then it will be possible to write a parser for the elf format, which should perform the same functions as the Engine.

However, examining the dart-sdk source code will still take some time. Recall that Dart is constantly under development, and the functionality of DartVM is no exception. It’s worth remembering that a parser written for a Snapshot is not universal. If the developer compiles its code using a newer version of Flutter, you will have to rewrite the parser to read the new libapp.so.

Another reading method can be used, which involves editing the source code and compiling the modified libflutter.so.

This will independently perform the necessary deserialization and read the instructions. Creating

source code patches from version to version is easier, while the Engine changes are not so drastic that they are hard to keep track of.

We've already learned how to compile libflutter.so; now we need to come up with some source code edits. For example, add print to the `Deserializer::ReadInstructions(CodePtr code, bool deferred)` function; as you may have guessed, this function is executed when an instruction is read from the _kDartIsolateSnapshotInstructions segment.

The Engine is additionally responsible for the network and file system. So, it would be useful to edit the `(Socket_CreateConnect) (Dart_NativeArguments args)` function, specifically `Socket::CreateConnect(addr)` by replacing addr with the address of our own proxy. Also, don't forget to disable certificate verification; you can introduce a patch: `bool ssl_crypto_x509_session_verify_cert_chain (SSL_SESSION *session) return true;`

It will now be possible to intercept the traffic of an analyzed application.

I created a utility reFlutter that modifies the source code, and configured CI/CD to compile Flutter Engine. Written in Python, it is intended for Flutter mobile applications. Supports Engine Android/iOS.

Let's take a closer look at how my utility works. It performs two main functions:

- Helps to compile Flutter Engine and introduce patches. (CI/CD, Docker, Local)
- Processes IPA/APK files. Retrieving Snapshot_hash from the libapp.so file. Downloading the file from the libflutter.so repository and further actions with it: replacing the preset IP of the network patch with a custom IP. Replacing the original libflutter.so in the target APK with the processed one.

reFlutter main logic is located in the _init_.py file. The ELFF(fname, **kwargs) function searches for Snapshot_hash in the libapp.so file. main() is searched for the required commit. The patchSource(hashS,ver) functions contain patches for the source code.

To differentiate versions, the enginehash.csv table was created with fields for matching Engine_commit (https://github.com/flutter/engine/blob/[Engine_commit]/DEPS) and Snasphot_hash (retrieved from the gen_snapshot file). The table is periodically updated with newly released versions of Flutter. When compiling Flutter Engine, we specify Snapshot_hash as an argument for reFlutter, and the required Engine_commit is retrieved from the table, which allows us to obtain the source code for the specific version of Flutter. The order of the fields in the table also matters. For hashes below line 28, for instance, a different patch is applied to the class_table.cc file.

reFlutter matches the Snapshot_hash from the libapp.so of analyzed IPA/APK file with the line in enginehash.csv; the user will see a message that the Engine version is not supported if the hash is not found.

- the frida.js script, which can be used to get function arguments by means of the resulting offset (after

running the patched APK/IPA file)
- the SNAPSHOT_HASH file contains the hash. Modifying it triggers the CI/CD and the start of the Engine compilation.
- the main.yml file contains the compilation script. Flutter Engine for Android uses ubuntu-18.04 and macos-11 for iOS.

The resulting Flutter Engines can be seen on the release page https://github.com/Impact-I/reFlutter/releases.

Libraries for Android support arm64, arm32 and iOS libraries support only arm64.

Supported builds: stable, beta. Dev is not supported—there are too many modifications, so the number of Snapshot_hash values for them would exceed 500. If desired, users can assemble the dev build themselves. Fortunately, developers don't usually use this branch, as it's unstable.

Now let's look at each applied patch in the source code.

First things first, the Flutter version examined here is 1.24.0-10.2.pre.

Let's start with the file: `src/third_party/dart/runtime/bin/socket.cc`.

This function is used to create a connection to the server. Our goal is to substitute the port and IP intended by the developer with our own proxy, enabling us to intercept traffic coming from the application.

```
void FUNCTION_NAME(Socket_CreateConnect)
(Dart_NativeArguments args) {
  RawAddr addr;

SocketAddress::GetSockAddr(Dart_GetNativeA
rgument(args, 1), &addr);
  Dart_Handle port_arg =
Dart_GetNativeArgument(args, 2);
  int64_t port =
DartUtils::GetInt64ValueCheckRange(port_ar
g, 0, 65535);

  + Syslog::PrintErr("ref:
%s",inet_ntoa(addr.in.sin_addr));
  + port=8083;
  + addr.addr.sa_family=AF_INET;
  + addr.in.sin_family=AF_INET;
  + inet_aton("192.168.133.104",
&addr.in.sin_addr);

  SocketAddress::SetAddrPort(&addr,
static_cast<intptr_t>(port));
  if (addr.addr.sa_family == AF_INET6) {
    Dart_Handle scope_id_arg =
Dart_GetNativeArgument(args, 3);
    int64_t scope_id =

DartUtils::GetInt64ValueCheckRange(scope_i
d_arg, 0, 65535);
    SocketAddress::SetAddrScope(&addr,
scope_id);
  }
  intptr_t socket =
Socket::CreateConnect(addr);
  OSError error;
```

For this implementation, we will simply overwrite the `addr` variable.

The reader is probably wondering: If we overwrite `addr` for the connection, how will the proxy know to which address send further requests?

Usually for HTTP proxy compatibility, the requests look like this:

```
GET http://example.org/index.php HTTP/1.1
Host: example.org
```

But simply by overwriting the address, as in the above patch, the URL will not contain the end host:

```
GET /index.php HTTP/1.1
Host: example.org
```

Therefore, to resolve this issue, Invisible Proxying is needed for intercepting traffic, which will take the destination address from the Host header.

But for those cases when the Host header in the application is incorrect, we will output the value of the `addr` variable before overwriting, using the following line `Syslog::PrintErr("ref: %s",inet_ntoa(addr.in.sin_addr))`.

Also remember to specify the AF_INET address family (IPv4).

The following patch is for the BoringSSL library (a fork of OpenSSL), which handles SSL.

This function checks the validity of the certificate chain; let's rewrite it to use any certificate in our proxy and always return true.

```
static bool
ssl_crypto_x509_session_verify_cert_chain(
SSL_SESSION *session,

SSL_HANDSHAKE *hs,

uint8_t *out_alert) {
  + return true;
  *out_alert = SSL_AD_INTERNAL_ERROR;
  STACK_OF(X509) *const cert_chain =
session->x509_chain;
  if (cert_chain == nullptr ||
sk_X509_num(cert_chain) == 0) {
    return false;
  }
```

Now let's analyze the patch which getting the offset of the functions `src/third_party/dart/runtime/vm/clustered_snapshot.cc`.

To deserialize libapp.so, the `void Deserializer::Deserialize(DeserializationRoots* roots)` function is run; this calls `Deserializer::ReadCluster()`, which reads clusters and initializes them as classes.

```
DeserializationCluster*
Deserializer::ReadCluster() {
intptr_t cid = ReadCid();
  Zone* Z = zone_;
  if (cid >= kNumPredefinedCids || cid ==
kInstanceCid) {
    return new (Z)
InstanceDeserializationCluster(cid);
  }
//…
  switch (cid) {
    case kClassCid:
      return new (Z)
ClassDeserializationCluster();
    case kTypeArgumentsCid:
      return new (Z)
TypeArgumentsDeserializationCluster();
    case kPatchClassCid:
      return new (Z)
PatchClassDeserializationCluster();
    case kFunctionCid:
      return new (Z)
FunctionDeserializationCluster();
    case kClosureDataCid:
      return new (Z)
ClosureDataDeserializationCluster();
    case kSignatureDataCid:
      return new (Z)
SignatureDataDeserializationCluster();
    case kRedirectionDataCid:
      return new (Z)
RedirectionDataDeserializationCluster();
    case kFfiTrampolineDataCid:
      return new (Z)
FfiTrampolineDataDeserializationCluster();
    case kFieldCid:
      return new (Z)
```

```
FieldDeserializationCluster();
    case kScriptCid:
      return new (Z)
ScriptDeserializationCluster();
    case kLibraryCid:
      return new (Z)
LibraryDeserializationCluster();
    case kNamespaceCid:
      return new (Z)
NamespaceDeserializationCluster();
    case kCodeCid:
      return new (Z)
CodeDeserializationCluster();
```

The structure is the following: `Library -> Class -> Function -> Code -> Instruction`.

Library can have more than one Class, and Class more than one Function. Function contains a Code object, which in turn contains the offset to the beginning of Instruction in the libapp.so file. Here's how it works.

```
class FunctionDeserializationCluster :
public DeserializationCluster {
  void ReadFill(Deserializer* d, bool
is_canonical) {
    Snapshot::Kind kind = d->kind();
    for (intptr_t id = start_index_; id <
stop_index_; id++) {
      FunctionPtr func =
static_cast<FunctionPtr>(d->Ref(id));
      Deserializer::InitializeHeader(func,
kFunctionCid,
     Function::InstanceSize());
      ReadFromTo(func);
```

```
      if (kind == Snapshot::kFullAOT) {
        func->ptr()->code_ =
static_cast<CodePtr>(d->ReadRef());
```

Further Code:

```
class CodeDeserializationCluster : public
DeserializationCluster {
  void ReadFill(Deserializer* d, intptr_t
id, bool deferred) {
    auto const code = static_cast<CodePtr>
(d->Ref(id));
    Deserializer::InitializeHeader(code,
kCodeCid, Code::InstanceSize(0));
    d->ReadInstructions(code, deferred);
```

Lastly, the function for reading instructions:

```
  if (FLAG_use_bare_instructions) {
    code->ptr()->instructions_ =
Instructions::null();
    previous_text_offset_ +=
ReadUnsigned();
    const uword payload_start =
        image_reader_-
>GetBareInstructionsAt(previous_text_offse
t_);
    const uint32_t payload_info =
ReadUnsigned();
    const uint32_t unchecked_offset =
payload_info >> 1;
```

```
    const bool has_monomorphic_entrypoint
= (payload_info & 0x1) == 0x1;

    const uword entry_offset =
has_monomorphic_entrypoint
                                    ?
Instructions::kPolymorphicEntryOffsetAOT
                                    0
    const uword monomorphic_entry_offset =
        has_monomorphic_entrypoint ?
Instructions::kMonomorphicEntryOffsetAOT
                                    0

    const uword entry_point =
payload_start + entry_offset;
    const uword monomorphic_entry_point =
        payload_start +
monomorphic_entry_offset;

    code->ptr()->entry_point_ =
entry_point;
    code->ptr()->unchecked_entry_point_ =
entry_point + unchecked_offset;
    code->ptr()->monomorphic_entry_point_
= monomorphic_entry_point;
    code->ptr()-
>monomorphic_unchecked_entry_point_ =
        monomorphic_entry_point +
unchecked_offset;
    return;
```

The `previous_text_offset_` variable stores the offset for our instruction. But we need to store this value and bind it to the required function. The challenge is making a small modification without breaking the compilation while maintaining compatibility with different Engine versions.Therefore, I made a rather

crude solution, which needs rewriting. But at the moment, the patch look like this:

```
code->ptr()->
>monomorphic_unchecked_entry_point_ =
        previous_text_offset_;
return;
```

The value is stored in the `monomorphic_unchecked_entry_point_` variable.

Next, let's consider the patches where the stored value is retrieved: `src/third_party/dart/runtime/vm/dart.cc`.

For better debugging, the developers created the FLAG_print_class_table flag; when set to true during the class table initialization stage, the names are output to the console.

Hence, we will replace the line in the function:

```
ErrorPtr Dart::InitializeIsolate(const
uint8_t* snapshot_data,
//…
  if (true) { // replace
(FLAG_print_class_table)
    I->class_table()->Print();
  }
```

Now let's switch directly to the called function, which already contains the patch: `src/third_party/dart/runtime/vm/class_table.cc`.

```
void ClassTable::Print()  {
+ OS::PrintErr("reFlutter");
+ char pushArr[160000]="";
  Class& cls = Class::Handle();
  String& name = String::Handle();
  for (intptr_t i = 1; i < top_; i++) {
    if (!HasValidClassAt(i)) {
      continue;
    }
    cls = At(i);
    if (cls.raw() != nullptr) {
      name = cls.Name();
   + auto& funcs =
Array::Handle(cls.functions());
//…
+   for (intptr_t c = 0; c <
funcs.Length(); c++) {
+         auto& func = Function::Handle();
+         func = cls.FunctionFromIndex(c);
+         String& signature =
String::Handle();
+         signature = func.Signature();
+         auto& codee =
Code::Handle(func.CurrentCode());
+         if(!func.IsLocalFunction()) {
+         strcat(classText," \n  ");
+
strcat(classText,func.ToCString());
+
+
strcat(classText,signature.ToCString());
```

```
+         strcat(classText," { \n\n
");
+         char append[70];
+         sprintf(append," Code Offset:
_kDartIsolateSnapshotInstructions +
0x%016" PRIxPTR
"\n",static_cast<uintptr_t>
(codee.MonomorphicUncheckedEntryPoint()));
//…
+         struct stat entry_info;
+         int exists = 0;
+         if (stat("/data/data/",
&entry_info)==0 &&
S_ISDIR(entry_info.st_mode)){
exists=1;         }
+         if(exists==1){
pid_t pid = getpid();
+         char path[64] = { 0 };
+         sprintf(path,
"/proc/%d/cmdline", pid);
+         FILE *cmdline = fopen(path,
"r");
+         if (cmdline) {
+         char chm[264] = { 0 };
char pat[264] = { 0 };     char
application_id[64] = { 0 };
+         fread(application_id,
sizeof(application_id), 1, cmdline);
+         sprintf(pat,
"/data/data/%s/dump.dart",
application_id);
+     do { FILE *f = fopen(pat, "a+");
```

First we get the name of the class: `name = cls.Name()`, then we get the function from the class: `func = cls.FunctionFromIndex(c)`.

We retrieve the Code object for the function: `auto& codee = Code::Handle(func.CurrentCode())`.

And now we obtain the previously stored **offset** from the `monomorphic_unchecked_entry_point_` variable:

```
sprintf(append," Code Offset:
_kDartIsolateSnapshotInstructions +
0x%016" PRIxPTR
"\n",static_cast<uintptr_t>
(codee.MonomorphicUncheckedEntryPoint())).
```

I have not listed the entire code, but I will highlight some of the features: retrieving the names of libraries, the names of classes and their interfaces, and the names of functions. Additionally, all extracted information is stored in the dump.dart file. Using `fread(application_id, sizeof(application_id), 1, cmdline)`, we retrieve the package name. To be able to use reFlutter on a non-root device, the `chmod(pat, S_IRWXU|S_IRWXG|S_IRWXO)` permissions are changed for the application internal folder.

On iOS, everything is implemented roughly the same way.

All patches are made, and as a result, we can intercept traffic and get the offset for functions.

What if you want to compile a dev build or create your own patch? For these purposes, I made a Docker image specifically for compiling the Flutter Engine: ptswarm/reflutter.

It supports only Android and uses ubuntu:18.04. Changes in the Flutter Engine code could be made

during a special pause. The source code is stored locally in the `/var/lib/docker/overlay2/<CONTAINER_ID>/merged/` container; the pause period is configured in the WAIT argument. E.g. `WAIT=300` allowing 5 minutes to change the Flutter Engine code.

The following sections will take a look at compiling with Docker.

Demo with an actual application

Let's analyze the security of a mobile application for Android and iOS written using Flutter without having the source code available.

The application we'll use is MyBMW.

We need to test:

- Backend (API Penetration Testing)
- Client side (Dart)

To test the API, we need to intercept application traffic. To check the client side for vulnerabilities, we need a method to perform static and dynamic analyses of the application.

Let's get to it!

```
impact@f:~$ pip install reflutter

impact@f:~$ reflutter
```

```
de.bmw.connected.mobile20.row.apk

Choose an option:

1. Traffic monitoring and interception
2. Display absolute code offset for
functions

[1/2]? 2

Example: (192.168.1.154) etc.

Please enter your BurpSuite IP:
192.168.1.64

SnapshotHash:
9cf77f4405212c45daf608e1cd646852

The resulting apk file: ./release.RE.apk
```

We chose option 2 since we needed to get a dump. However, if we only need to intercept traffic, option 1 is better. Dumping functionality loads an application and slows it down, which makes it difficult to use, especially on old devices.

Next, we need to sign the APK. I recommend using uber-apk-signer, because it works better than other utilities:

```
impact@f:~$ java -jar uber-apk-signer.jar
--allowResign -a release.RE.apk

VERIFY

file: release.RE-aligned-debugSigned.apk
```

```
(509.1 MiB)

checksum:
13af6240e23b5f79dc51b9eae8b9a987a67a0ea517
aa2feda40ed50dd93632f8 (sha256)

- zipalign verified

- signature verified [v1, v2, v3]
```

Finally, we install the signed APK on the device:

```
adb install release.RE-aligned-
debugSigned.apk
```

Now we start the application:

Hopefully, our libflutter.so has already read the instructions. We check and retrieve dump.dart.

```
adb -d shell "cat
/data/data/de.bmw.connected.mobile20.row/d
```

```
ump.dart" > dump.dart
```

We view its contents:

```
~$ nano dump.dart
//…
Library:'package:remote_cameras/src/reposi
tory/bmw_crypto/bmw_crypto.dart' Class:
Aes extends Object {
  AesCbc* aesCbc = sentinel ;

  Function 'Aes._@10765229738':
constructor. String: null {

              Code Offset:
_kDartIsolateSnapshotInstructions +
0x000000000000260c

      }

  Function 'Aes.': static factory. String:
null {

              Code Offset:
_kDartIsolateSnapshotInstructions +
0x000000000139e774

      }

  Functions String: null {

              Code Offset:
_kDartIsolateSnapshotInstructions +
0x000000000139e3b8
```

```
        }

      }
Library:'package:user_repository/src/api/authentication/models/authentication_api_endpoints.dart' Class: AuthenticationApiEndpointsApim extends Object {
  String deleteToken = eadrax-coas/v1/oauth/token ;
  String postToken = eadrax-coas/v1/oauth/token ;
  String postTokenIdentifier = eadrax-coas/v1/oauth/token/identifier ;
  String smsCnLogin = eadrax-coas/v2/login/sms ;
  String sendCnSmsVerificationCode = eadrax-coas/v1/cop/message ;
  String postCnToken = eadrax-coas/v2/login/pwd ;
  String isSliderCaptchaNeeded = eadrax-coas/v2/cop/is-captcha-needed ;
  String postSliderCaptcha = eadrax-coas/v2/cop/slider-captcha ;
  String postCheckCaptcha = eadrax-coas/v1/cop/check-captcha ;
  String postCnGuestToken = eadrax-coas/v1/glogin ;
  String postBindWechat = eadrax-coas/v2/cop/wechat/bind ;
  String getUnBindWechat = eadrax-coas/v2/cop/wechat/unbind ;
  String postLoginWithWechat = eadrax-coas/v2/login/wechat ;
  String postBindAppleId = eadrax-coas/v2/cop/apple/bind ;
  String getUnBindAppleId = eadrax-coas/v2/cop/apple/unbind ;
  String postLoginWithAppleId = eadrax-
```

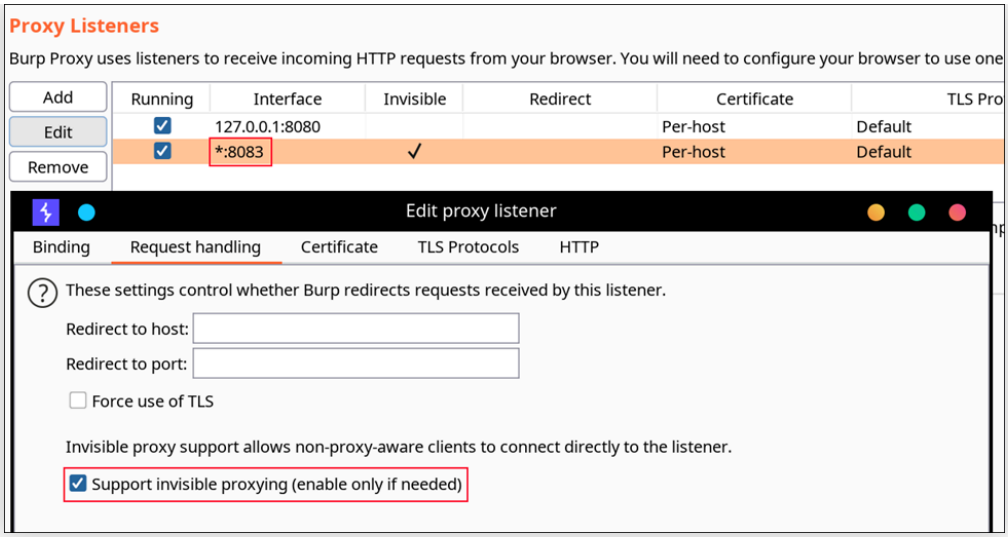
```
coas/v2/login/apple ;
  String postBindWechatAndLogin = eadrax-
coas/v2/wechat/bind/sms ;
  String postBindAppleIdAndLogin = eadrax-
coas/v2/apple/bind/sms ;
  String getWeChatInfo = eadrax-
coas/v2/cop/wechat/info ;
  String postHkToken = eadrax-
hkcos/v2/connected/login/pwd/nonce ;
  String sendHkSmsVerificationCode =
eadrax-hkcos/v1/connected/forgetpassword ;
  String postTokenHk = eadrax-
hkcos/v1/oauth/token ;
  String deleteTokenHk = eadrax-
hkcos/v1/oauth/token ;
  String postTokenCn = eadrax-
coas/v2/oauth/token ;

      }
```

Ok, it looks like the developers are building the application without the obfuscate flag, and we can see the original names of the libraries, classes, and functions.

You can see that the `AuthenticationApiEndpointsApim` class contains the API endpoints for the authentication function.

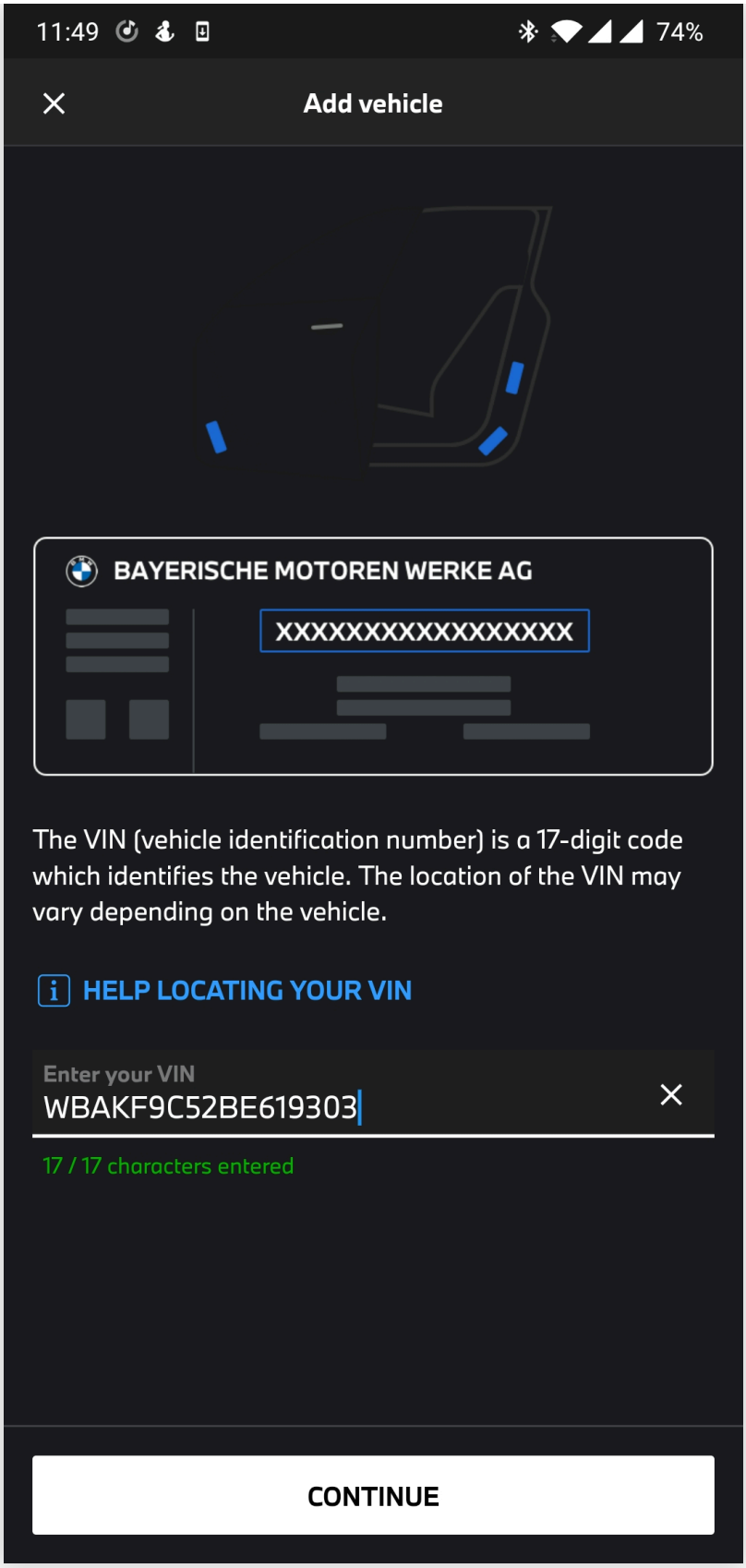
Now let's try to intercept the traffic.

We need to use Invisible Proxying. I recommend using BurpSuite, which supports this mode.

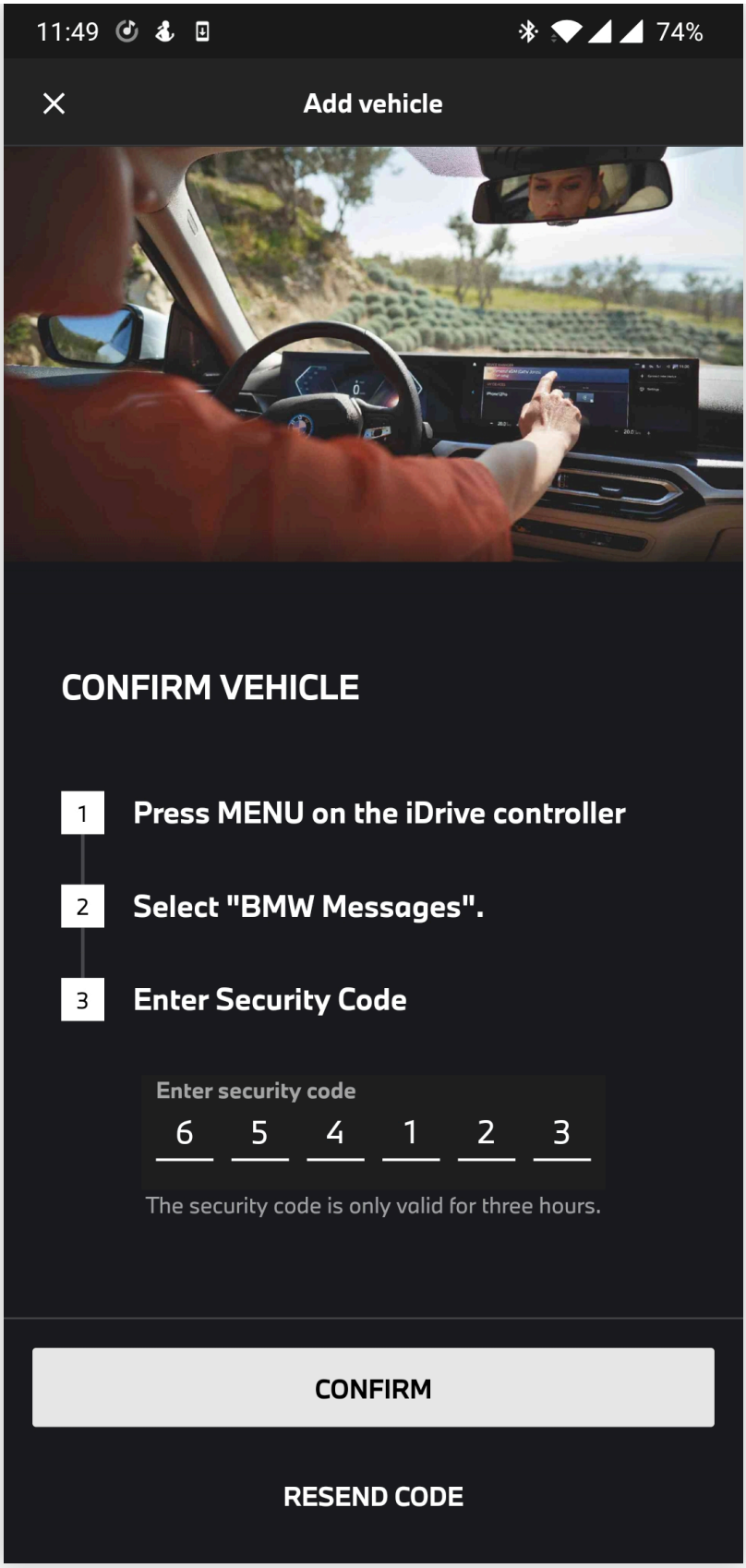


We need to select All Interfaces on the Binding tab, enable Invisible Proxying and specify port 8083.

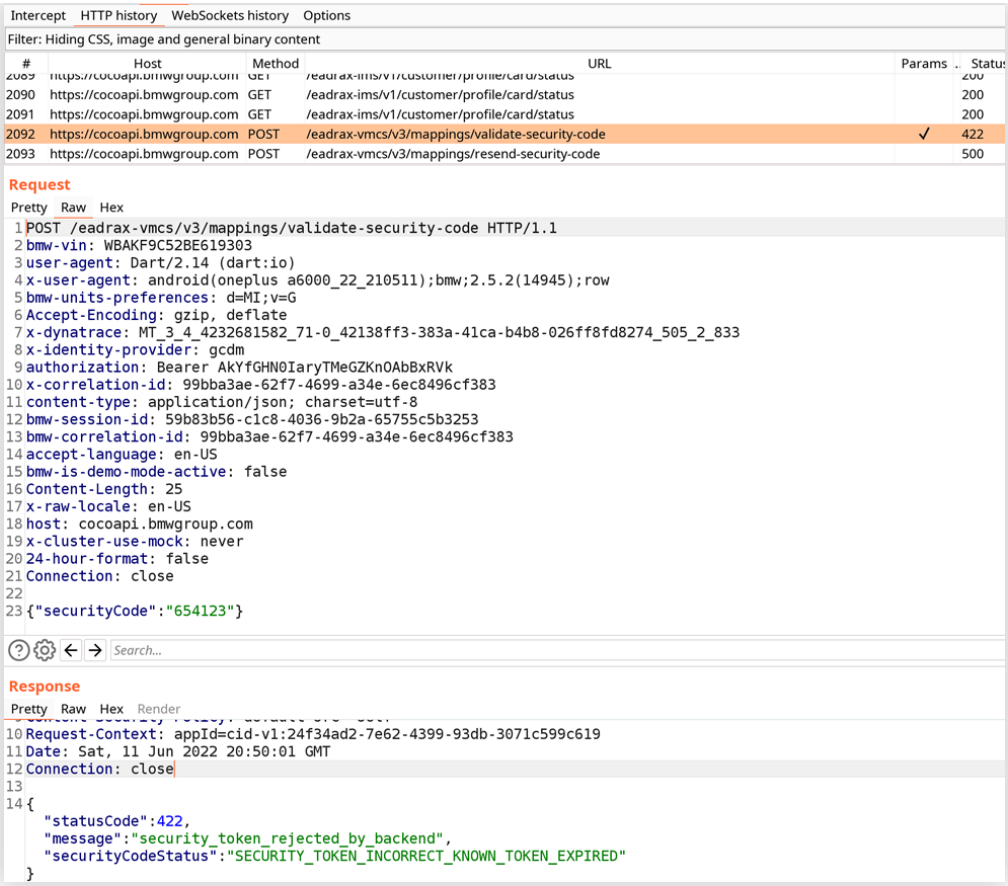
When the MyBMW application runs, traffic appears on the Proxy tab. Let’s intercept some requests. For the test, we use vehicle-binding functionality using a vehicle identification number (VIN).



After clicking Continue, we enter the Security Code:



Let’s look at the received requests on the Proxy tab:



The requests go to the cocoapi.bmwgroup.com host. In the POST request, we see the entered VIN and Security Code; let's find out the according function in the dump.dart file.

```
Library:'package:vehicle_mapping_repository/src/api/vehicle_mapping_api_client.dart'
Class: VehicleMappingApiClient extends Object {
//…
Functions String: null {

                Code Offset: _kDartIsolateSnapshotInstructions + 0x00000000019c2850

        }
  Functions String: null {
```

```
                Code Offset:
_kDartIsolateSnapshotInstructions +
0x00000000019c2c54

        }
```

We can assume that the last function checks the code by sending it to the server. Let's use the Frida script to capture the function arguments. But first, we need to get the value of `_kDartIsolateSnapshotInstructions`.

```
impact@f:~$ readelf -Ws ./libapp.so
Symbol table '.dynsym' contains 6 entries:
   Num:    Value          Size Type
Bind   Vis      Ndx Name
     0: 0000000000000000     0 NOTYPE
LOCAL  DEFAULT  UND
     1: 00000000025cc000 19104 OBJECT
GLOBAL DEFAULT    7
_kDartVmSnapshotInstructions
     2: 00000000025d0aa0 0x2c5a060 OBJECT
GLOBAL DEFAULT    7
_kDartIsolateSnapshotInstructions
     3: 00000000000001b0 30192 OBJECT
GLOBAL DEFAULT    2 _kDartVmSnapshotData
     4: 00000000000077a0 0x25c08a0 OBJECT
GLOBAL DEFAULT    2
_kDartIsolateSnapshotData
     5: 0000000000000190    32 OBJECT
GLOBAL DEFAULT    1 _kDartSnapshotBuildId
```

Ok, it's 25d0aa0. Now it remains to add these two CodeOffset values: 25d0aa0 + 00000000019c2c54. We get 3F936F4.

We modify the Frida script by entering the correct offset.

```
function hookFunc() {
    var dumpOffset = '0x3F936F4' //
_kDartIsolateSnapshotInstructions + code
offset

    var argBufferSize = 150
    var address =
Module.findBaseAddress('libapp.so') //
libapp.so (Android) or App (IOS)
    console.log('\n\nbaseAddress: ' +
address.toString())
```

Great. We run the script and click "ADD MY BMW".

```
impact@f:~$ frida -U -f
de.bmw.connected.mobile20.row -l frida.js
--no-pause

//…

Argument 2 address 0x7683639c99 buffer:
150

 Value:
00000000  02 52 00 00 00 00 00 0c 00 00 00
00 00 00 00 36  .R.............6
```

```
00000010  35 34 31 32 33 00 00 41 80 28 e3
76 00 00 00 04  54123..A.(.v....
00000020  03 4f 00 00 00 00 00 41 80 28 e3
76 00 00 00 06  .O.....A.(.v....
00000030  00 00 00 00 00 00 00 91 4f e0 46
77 00 00 00 b1  ........O.Fw....
00000040  bf 35 7c 76 00 00 00 a1 91 30 7c
76 00 00 00 04  .5|v.....0|v....
00000050  05 4f 00 00 00 00 00 b1 5f 4b be
76 00 00 00 0c  .O......_K.v....
00000060  00 00 00 00 00 00 00 f1 0d d5 ed
76 00 00 00 b1  ...........v....
00000070  01 d5 ed 76 00 00 00 f1 26 d5 ed
76 00 00 00 d1  ...v....&..v....
00000080  d5 d4 ed 76 00 00 00 81 15 d5 ed
76 00 00 00 11  ...v.......v....
00000090  26 d5 ed 76 00 00
&..v..

________________________________________
_________

Argument 4 address 0x7794f9b6c1 buffer:
150

 Value:
00000000  03 52 00 00 00 00 00 22 00 00 00
00 00 00 00 57  .R.....".......W
00000010  42 41 4b 46 39 43 35 32 42 45 36
31 39 33 30 33  BAKF9C52BE619303
00000020  80 28 e3 76 00 00 00 41 80 28 e3
76 00 00 00 1a  .(.v...A.(.v....
00000030  04 7a 00 00 00 00 00 08 b7 f9 94
77 00 00 00 10  .z.........w....
00000040  00 00 00 00 00 00 00 00 00 00 00
ec 5d 18 dd 00  ............]...
00000050  00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00  ................
00000060  00 00 00 00 00 00 00 00 00 00 00
```

```
00 00 00 00 1a  .................
00000070  04 4d 00 00 00 00 00 81 58 48 be
76 00 00 00 fe  .M......XH.v....
00000080  ff ff 7f 00 00 00 00 a1 04 30 7c
76 00 00 00 04  .........0|v....
00000090  00 00 00 00 00 00
......
```

We've intercepted the `validateVehicleSecurityCode` function. Argument 2 contains Security Code 654123. Argument 4 contains the VIN WBAKF9C52BE619303.
Let's look at another functionality, for example, PIN processing, and see how the application stores it.
We go back to our dump.dart file.

```
Library:'package:user_repository/src/user_
repository.dart' Class:
ConnectedUserRepository extends Object
implements Type: UserRepository {
//…
  Functions String: null {

               Code Offset:
_kDartIsolateSnapshotInstructions +
0x000000000125aa0c

       }
```

The `savePin` function is called when a PIN added/changed. We modify the Frida script by entering the correct offset.

```
function hookFunc() {
    var dumpOffset = '0x382B4AC' //
_kDartIsolateSnapshotInstructions + code
offset
```

We go to Settings in My BMW and click “PIN CHANGE”.

```
frida -U -f de.bmw.connected.mobile20.row
-l frida.js --no-pause

//…

Argument 3 address 0x7bd29c5f29 buffer:
150

 Value:
00000000  00 00 00 00 00 00 00 08 00 00 00
00 00 00 00 31  ................1
00000010  36 35 34 7e 00 00 00 41 80 e0 35
7e 00 00 00 04  654.............
00000020  04 36 5c 00 00 00 00 08 00 00 00
00 00 00 00 08  ................
00000030  00 00 00 00 00 00 00 08 00 00 00
00 00 00 00 08  ................
00000040  00 00 00 00 00 00 00 b1 7e be 2a
7e 00 00 00 71  ........~.*~...q
00000050  80 e0 35 7e 00 00 00 41 80 e0 35
7e 00 00 00 04  ..5~...A..5~....
00000060  02 85 10 00 00 00 00 29 5f 9c d2
7b 00 00 00 49  .......)_..{...I
00000070  5f 9c d2 7b 00 00 00 99 54 9c d2
7b 00 00 00 04  _..{....T..{....
00000080  02 e3 5e 00 00 00 00 b1 5f bb 2f
```

```
7e 00 00 00 29  ..^.....)
00000090  5f 9c d2 7b 00 00
_..{..
```

We've intercepted the `savePin` function. Argument 3 contains the entered code 1654, which is encrypted using AES and then saved to the file `/data/data/de.bmw.connected.mobile20.row/shared_prefs/FlutterSecureStorage.xml`

File contents:

```
<?xml version='1.0' encoding='utf-8'
standalone='yes' ?>
<map>
//…
    <string
name="VGhpcyBpcyB0aGUgcHJlZml4IGZvciBhIHNl
Y3VyZSBzdG9yYWdlCg_is_jailbreak_warning_di
sabled">GwD3z9NtRtuR5PaaluteWOWu9w95ARi2d4
hfaTxkhLw=&#10;    </string>
    <string
name="VGhpcyBpcyB0aGUgcHJlZml4IGZvciBhIHNl
Y3VyZSBzdG9yYWdlCg_analytics_toggle">YIdG4
oT75cjbNXsHMqxVzXbgAHRR0KwS1Sz69mKB2e8=&#1
0;    </string>
    <string
name="VGhpcyBpcyB0aGUgcHJlZml4IGZvciBhIHNl
Y3VyZSBzdG9yYWdlCg_access_token">fitsdsrm9
XPZc7CZ78ooVZUP8F/svUQX9a9JN5mFV9d10JpCkE0
M04ghliP5TMUA&#10;    </string>
    <string
name="VGhpcyBpcyB0aGUgcHJlZml4IGZvciBhIHNl
Y3VyZSBzdG9yYWdlCg_pin">O0b6BJm9kRWchaSmuf
```

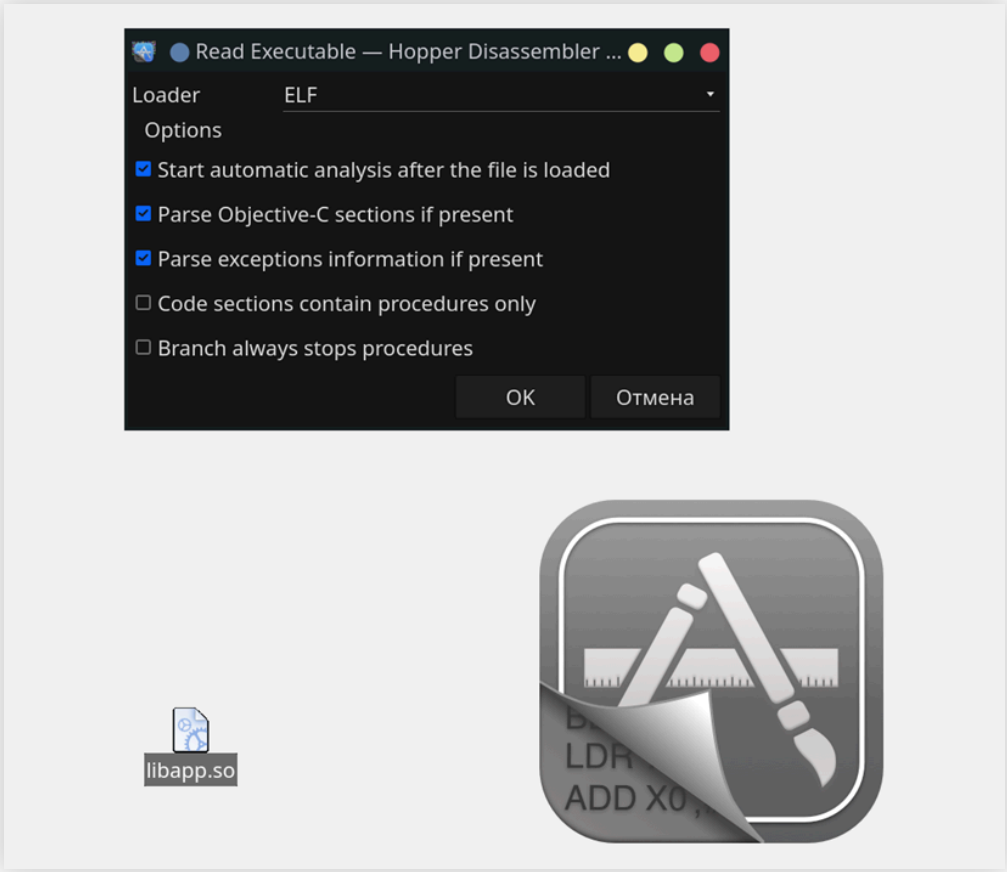
```
93JGoNXrVQT3XTVPFppabso6g=&#10;
</string>
```

There are fields like _pin and _access_token. Let's see how these values are encrypted. In dump.dart, we find a class with a name similar to FlutterSecureStorage.xml:

```
Library:'package:flutter_secure_storage/fl
utter_secure_storage.dart' Class:
FlutterSecureStorage extends Object {
  FlutterSecureStoragePlatform
_platform@1401243328 = sentinel ;

  Function 'write':. String: null {

              Code Offset:
_kDartIsolateSnapshotInstructions +
0x000000000029a7a4

      }

  Function 'read':. String: null {

              Code Offset:
_kDartIsolateSnapshotInstructions +
0x000000000029a248

      }
  Function '_selectOptions@1401243328':.
String: null {

              Code Offset:
_kDartIsolateSnapshotInstructions +
0x000000000029a920
```

```
}
```

Let’s try to analyze the functions statically using Hopper.



We import the library `unzipBmwApk/lib/arm64-v8a/libapp.so`.

To find the write function, we calculate 25d0aa0 + 000000000029a7a4 and move to the procedure at 0x286B244.

```
        ; =============== B E G I N N I N G   O F   P R O C E D U R E ===============


                        sub_286b244:
000000000286b244            stp         fp, lr, [x15, #-0x10]!                  ; CODE XREF=sub_286bec8+512
000000000286b248            mov         fp, x15
000000000286b24c            sub         x15, x15, #0x8
000000000286b250            ldr         x16, [x26, #0x40]
000000000286b254            cmp         x15, x16
000000000286b258            b.ls        loc_286b32c

                        loc_286b25c:
000000000286b25c            ldr         x0, [fp, #0x10]                         ; CODE XREF=sub_286b244+236
000000000286b260            cmp         x0, x22
000000000286b264            b.ne        loc_286b2c4

000000000286b268            ldr         x0, [x26, #0x88]
000000000286b26c            ldr         x0, [x0, #0x19b0]
000000000286b270            ldr         x16, [x27, #0x28]
000000000286b274            cmp         x0, x16
000000000286b278            b.ne        loc_286b284

000000000286b27c            ldr         x0, [x27, #0x6e30]                      ; argument #1 for method sub_521ee98
000000000286b280            bl          sub_521ee98                             ; sub_521ee98

                        loc_286b284:
000000000286b284            stur        x0, [fp, #-0x8]                         ; CODE XREF=sub_286b244+52
000000000286b288            ldr         x16, [fp, #0x28]
000000000286b28c            ldr         lr, [fp, #0x20]
000000000286b290            stp         lr, x16, [x15, #-0x10]!
000000000286b294            bl          sub_286b3c0                             ; sub_286b3c0
000000000286b298            add         x15, x15, #0x10
000000000286b29c            mov         x1, x0
000000000286b2a0            ldur        x0, [fp, #-0x8]
000000000286b2a4            cmp         x0, x22
000000000286b2a8            b.eq        loc_286b334

000000000286b2ac            ldr         x16, [fp, #0x18]
000000000286b2b0            stp         x16, x0, [x15, #-0x10]!
000000000286b2b4            str         x1, [x15, #-0x8]!
000000000286b2b8            bl          sub_286b068                             ; sub_286b068
000000000286b2bc            add         x15, x15, #0x18
000000000286b2c0            b.al        loc_286b320
```


The sub_286af58 function is called inside the procedure.

```
                        loc_286b2c4:
000000000286b2c4            ldr         x0, [x26, #0x88]                        ; CODE XREF=sub_286b244+32
000000000286b2c8            ldr         x0, [x0, #0x19b0]
000000000286b2cc            ldr         x16, [x27, #0x28]
000000000286b2d0            cmp         x0, x16
000000000286b2d4            b.ne        loc_286b2e0

000000000286b2d8            ldr         x0, [x27, #0x6e30]                      ; argument #1 for method sub_521ee98
000000000286b2dc            bl          sub_521ee98                             ; sub_521ee98

                        loc_286b2e0:
000000000286b2e0            stur        x0, [fp, #-0x8]                         ; CODE XREF=sub_286b244+144
000000000286b2e4            ldr         x16, [fp, #0x28]
000000000286b2e8            ldr         lr, [fp, #0x20]
000000000286b2ec            stp         lr, x16, [x15, #-0x10]!
000000000286b2f0            bl          sub_286b3c0                             ; sub_286b3c0
000000000286b2f4            add         x15, x15, #0x10
000000000286b2f8            mov         x1, x0
000000000286b2fc            ldur        x0, [fp, #-0x8]
000000000286b300            cmp         x0, x22
000000000286b304            b.eq        loc_286b338

000000000286b308            ldr         x16, [fp, #0x18]
000000000286b30c            stp         x16, x0, [x15, #-0x10]!
000000000286b310            ldr         x16, [fp, #0x10]
000000000286b314            stp         x16, x1, [x15, #-0x10]!
000000000286b318            bl          sub_286af58                             ; sub_286af58
000000000286b31c            add         x15, x15, #0x20

                        loc_286b320:
000000000286b320            mov         x15, fp                                 ; CODE XREF=sub_286b244+124
000000000286b324            ldp         fp, lr, [x15], #0x10
000000000286b328            ret
                          ; endp

                        loc_286b32c:
000000000286b32c            bl          sub_52209b0                             ; sub_52209b0, CODE XREF=sub_286b244+20
000000000286b330            b.al        loc_286b25c

                        loc_286b334:
000000000286b334            bl          sub_52211d8                             ; sub_52211d8, CODE XREF=sub_286b244+100
                          ; endp

                        loc_286b338:
000000000286b338            bl          sub_52211d8                             ; sub_52211d8, CODE XREF=sub_286b244+192
                          ; endp
```

We find the function in dump.dart: 286af58 – 25d0aa0

```
Library:'package:flutter_secure_storage_pl
atform_interface/flutter_secure_storage_pl
```

```
atform_interface.dart' Class:
MethodChannelFlutterSecureStorage extends
FlutterSecureStoragePlatform {
  Function 'write':. String: null {

                Code Offset:
_kDartIsolateSnapshotInstructions +
0x000000000029a4b8

        }
```

Looks like a function from the `MethodChannelFlutterSecureStorage` class is being called in FlutterSecureStorage.

Let's move on to `sub_286af58`.

```
      ; ================ B E G I N N I N G   O F   P R O C E D U R E ================


                    sub_286af58:
000000000286af58        stp         fp, lr, [x15, #-0x10]!                  ; CODE XREF=sub_286b244+212
000000000286af5c        mov         fp, x15
000000000286af60        ldr         x16, [x26, #0x40]
000000000286af64        cmp         x15, x16
000000000286af68        b.ls        loc_286afe4
--------------------------------------------------------------------------------
                    loc_286af6c:
000000000286af6c        mov         x1, x22                                 ; argument #2 for method sub_522076c, CODE XREF=sub_286af58+144
000000000286af70        mov         x2, #0xc                                ; argument #3 for method sub_522076c
000000000286af74        bl          sub_522076c                             ; sub_522076c
000000000286af78        ldr         x17, [x27, #0x3090]
000000000286af7c        stur        x17, [x0, #0x17]
000000000286af80        ldr         x1, [fp, #0x20]
000000000286af84        stur        x1, [x0, #0x1f]
000000000286af88        ldr         x17, [x27, #0x6e40]
000000000286af8c        stur        x17, [x0, #0x27]
000000000286af90        ldr         x1, [fp, #0x10]
000000000286af94        stur        x1, [x0, #0x2f]
000000000286af98        ldr         x17, [x27, #0x6e48]
000000000286af9c        stur        x17, [x0, #0x37]
000000000286afa0        ldr         x1, [fp, #0x18]                         ; argument #2 for method sub_25d9e34
000000000286afa4        stur        x1, [x0, #0x3f]
000000000286afa8        ldr         x16, [x27, #0x1f78]
000000000286afac        stp         x0, x16, [x15, #-0x10]!
000000000286afb0        bl          sub_25d9e34                             ; sub_25d9e34
000000000286afb4        add         x15, x15, #0x10
000000000286afb8        ldr         x16, [x27, #0x390]
000000000286afbc        ldr         lr, [x27, #0x6e50]
000000000286afc0        stp         lr, x16, [x15, #-0x10]!
000000000286afc4        ldr         x16, [x27, #0x1010]
000000000286afc8        stp         x0, x16, [x15, #-0x10]!
000000000286afcc        ldr         x4, [x27, #0x438]
000000000286afd0        bl          sub_27e4644                             ; sub_27e4644
000000000286afd4        add         x15, x15, #0x20
000000000286afd8        mov         x15, fp
000000000286afdc        ldp         fp, lr, [x15], #0x10
000000000286afe0        ret
                      ; endp
```

The 27e4644 function is called inside the procedure; we find it in dump.dart

```
Library:'package:flutter/src/services/plat
form_channel.dart' Class: MethodChannel
```

```
extends Object {
  Function 'invokeMethod':. String: null {

              Code Offset:
_kDartIsolateSnapshotInstructions +
0x0000000000213ba4

       }
```

The `MethodChannel` class is part of Flutter's standard libraries; the `invokeMethod` can be used to call functions implemented in Java (dex).

The calls take place in the following order: (FlutterSecureStorage) write -> (MethodChannelFlutterSecureStorage) write -> (MethodChannel) invokeMethod -> classes.dex (Java Code)
We decompile classes.dex using Jadx and find the `FlutterSecureStoragePlugin` class.

```
/* renamed from: g.k.a.d */
public class FlutterSecureStoragePlugin
implements
MethodChannel.MethodCallHandler,
FlutterPlugin {
 class RunnableC8275b implements Runnable
{   public void run() {
            try {
                String str =
this.f26082a.method;
                char c = 65535;
                switch (str.hashCode()) {
                    case -1335458389:
```

```
                        if
(str.equals("delete")) {   c = 4;  break;
}    break;
                    case -358737930:
                        if
(str.equals("deleteAll")) {  c = 5;
break;  }   break;
                    case 3496342:
                        if
(str.equals("read")) {  c = 1;   break;
}  break;
                    case 113399775:
                        if
(str.equals("write")) {  c = 0;  break;  }
break;
                    case 208013248:
                        if
(str.equals("containsKey")) {  c = 3;
break; }
                        break;
                    case 1080375339:
                        if
(str.equals("readAll")) {  c = 2;  break;
}
                        break; //…
    @Override //
p465io.flutter.plugin.common.MethodChannel
.MethodCallHandler
    public void onMethodCall(MethodCall
methodCall, MethodChannel.Result result) {
        this.f26077h.post(new
RunnableC8275b(methodCall, new
C8274a(result)));
    }
```

The Dart function (`MethodChannel`) `invokeMethod` calls `onMethodCall(MethodCall methodCall, MethodChannel.Result result)`, and passes a

serialized string in the format: `{method: write, {options={resetOnError=false, encryptedSharedPreferences=false}, value=$value, key=$key}}`

Let's put together a Frida script for the Java method `onMethodCall`.

```
setTimeout(function() {

    Java.perform(function() {
        let FlutterSecureStoragePlugin =
Java.use("g.k.a.d");

FlutterSecureStoragePlugin.onMethodCall.ov
erload('io.flutter.plugin.common.MethodCal
l',
'io.flutter.plugin.common.MethodChannel$Re
sult').implementation =
function(MethodCall, sentinel) {
            let ret =
Java.cast(MethodCall.getClass(),
Java.use("java.lang.Class")).getDeclaredFi
eld("method");
            let values =
ret.get(MethodCall);
            console.log('onMethodCall:
method: ' + values);

            ret =
Java.cast(MethodCall.getClass(),
Java.use("java.lang.Class")).getDeclaredFi
eld("arguments");
            values = ret.get(MethodCall);
            console.log('onMethodCall:
values: ' + values);
                        console.log();
```

```
                    return
FlutterSecureStoragePlugin.onMethodCall.ov
erload('io.flutter.plugin.common.MethodCal
l',
'io.flutter.plugin.common.MethodChannel$Re
sult').call(this, MethodCall, sentinel);
            };
        });
}, 0);
```

And run it:

```
impact@f:~$ frida -U -f
de.bmw.connected.mobile20.row -l
fridasnippet.js --no-pause
//…
onMethodCall: method: write
onMethodCall: values: {options=
{resetOnError=false,
encryptedSharedPreferences=false},
value=alcpMOwBOW5NsNZxVcDD69NkNpc,
key=access_token}
onMethodCall: method: write
onMethodCall: values: {options=
{resetOnError=false,
encryptedSharedPreferences=false},
value=6543, key=pin}
```

FlutterSecureStorage is a popular library used in half of all Flutter applications. Source code and description are available here: pub.dev/packages/flutter_secure_storage.

You can use this Frida script in Flutter applications for security analysis by changing the name of the class (g.k.a.d).

Recompile the Engine using Docker

Flutter is developed by Google, which naturally uses it to create its own mobile apps. For example, Google Ads or Google Pay.

The company's developers prefer the **dev** branch for release. And since reFlutter only supports stable and beta, to read libapp.so you'll have to compile the Engine yourself.

```
Library:'package:nbu.paisa.gpay.database.conversation/src/model/conversation_card_conversation.dart' Class: _$ConversationCardConversation@10672202543 extends ConversationCardConversation {

  Function 'get:id': getter const. (_$ConversationCardConversation@10672202543) => int? {
              Code Offset: _kDartIsolateSnapshotInstructions + 0x00000000015a6248
       }
      }
Library:'package:nbu.paisa.gpay.url_launcher/src/url_launcher.dart' Class: UrlLaunchException extends Object implements Type: Exception {
```

```
  Function 'get:message': getter const.
(UrlLaunchException) => String {
              Code Offset:
_kDartIsolateSnapshotInstructions +
0x0000000001608188
       }
      }
```

For such cases, it's possible to use the specially created Docker image, in which I automatically apply patches using reFlutter. You can also add your own patches.

As a test, I'll use a standard Flutter-based application but compile it with the **dev** Engine.

```
~/AndroidStudio/flutter/bin$ flutter
channel dev
```

We switch the channel and then compile the application.

```
impact@f:~$ reflutter
./AndroidStudioProjects/ptswarm2/build/app
/outputs/flutter-apk/app-release.apk

Engine SnapshotHash:
e8b7543ba0865c5bac45bf158bb3d4c1

This engine is currently not supported.

Most likely this flutter application uses
```

```
the Debug version engine which you need to
build manually using Docker at the moment.
```

We apply reFlutter on the resulting APK and see a message that there is no hash in the enginehash.csv table.

We have snapshothash but for the compilation we need a commit. To find it, I wrote the small script ./searchSnapshot.sh.

It works as follows:

- Creates a Flutter folder.
- Retrieves all commits.
- Downloads the required gen_snapshot from the storage.googleapis.com server for each commit.
- Extracts the following fields: Dart SDK version, Engine Commit, EngineHashSnapshot, into the ./flutter/ListEngine.info file.

After running the script, we need to find the previously obtained SnapshotHash in the ListEngine.info file.

For e8b7543ba0865c5bac45bf158bb3d4c1, we get these fields:

```
Dart SDK version: 2.13.0-30.0.dev (dev)
(Fri Feb 12 04:33:47 2021 -0800) on
"linux_simarm64"
Engine:
6993cb229b99e6771c6c6c2b884ae006d8160a0f
```

```
EngineHashSnapshot:
e8b7543ba0865c5bac45bf158bb3d4c1
```

Now the compilation can start, but reFlutter needs the closest supported SnasphotHash to apply the correct patches.

Let's open this commit https://github.com/flutter/engine/tree/6993cb229b99e6771c6c6c2b884ae006d8160a0f.

The date is on 16 Feb 2021.

We find the nearest date in https://github.com/flutter/flutter/tags?after=1.26.0-17.8.pre.

For this date, the most recent Flutter version is 1.27.0-4.0.pre.

We open the enginehash.csv table and search for the nearest SnasphotHash by version. It seems to be between 1.25.0-8.3.pre and 2.0.6.

version	Engine_commit
....	....
2.0.6	05e680e202af9a92461070cb2d9982acad4
1.25.0-8.3.pre	7a8f8ca02c276dce02f8dd42a44e776ac03f

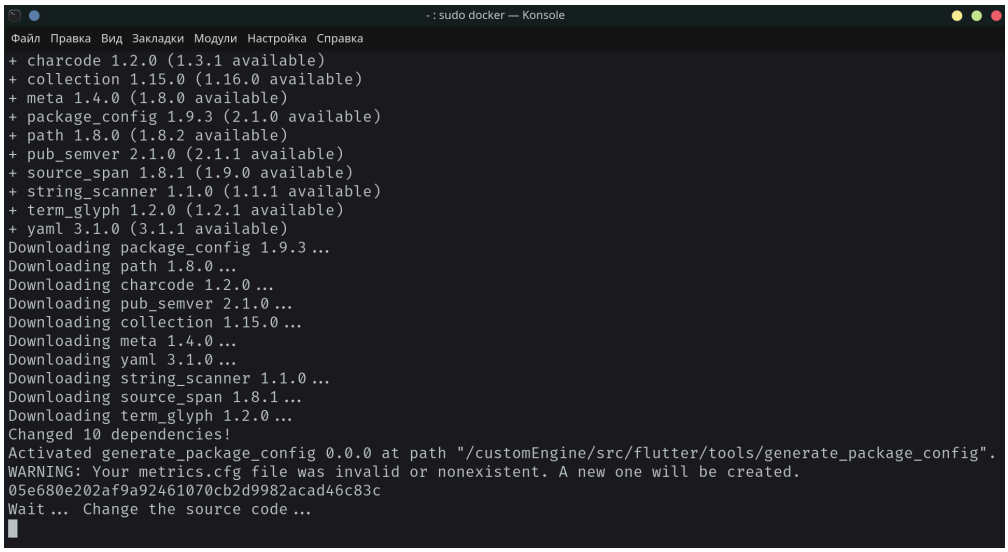
We take 5b97292b25f0a715613b7a28e0734f77 as a guess.

Now we can use Docker to compile the **dev** engine.

```
sudo docker run -e WAIT=1000 -e x64=0 -e
arm=0 -e
HASH_PATCH=5b97292b25f0a715613b7a28e0734f7
7 -e
COMMIT=9bcb3bfb0ecbc0ec763ade5f19dd1aa65e8
8e579 --rm -iv${PWD}:/t ptswarm/reflutter
```

HASH_PATCH specifies SnasphotHash to search for the required patch.
COMMIT is the compiled Engine commit.
-e x64 -e arm is set to 0 if you don't want to compile any of the architectures.
WAIT specifies the time in seconds to wait for the applied patches.

After starting Docker, a wait message appears after some time.



The time allowed to edit and review the applied patches is 1000 seconds.

The source code can be found and modified in the Docker container. In my case, it is `/var/lib/docker/overlay2/<CONTAINER_ID>/merged/customEngine/src/`.

At the time of writing, patches were automatically applied in the following folders:

```
/var/lib/docker/overlay2/<CONTAINER_ID>/merged/customEngine/src/third_party/dart/runtime/vm/
/var/lib/docker/overlay2/<CONTAINER_ID>/merged/customEngine/src/third_party/dart/tools/
/var/lib/docker/overlay2/<CONTAINER_ID>/merged/customEngine/src/third_party/boringssl/src/ssl/
/var/lib/docker/overlay2/<CONTAINER_ID>/merged/customEngine/src/third_party/dart/runtime/bin/
```

Here are the files:

Let’s take a look at the changes made in the socket.cc file:

```
361
362 void FUNCTION_NAME(Socket_CreateConnect)(Dart_NativeArguments args) {
363   RawAddr addr;
364   SocketAddress::GetSockAddr(Dart_GetNativeArgument(args, 1), &addr);
365   Dart_Handle port_arg = Dart_GetNativeArgument(args, 2);
366   int64_t port = DartUtils::GetInt64ValueCheckRange(port_arg, 0, 65535);
367   Syslog::PrintErr("ref: %s",inet_ntoa(addr.in.sin_addr));
368   if(port>50){port=8083;addr.addr.sa_family=AF_INET;addr.in.sin_family=AF_INET;
369       inet_aton("192.168.133.104", &addr.in.sin_addr);}
370   SocketAddress::SetAddrPort(&addr, static_cast<intptr_t>(port));
371   if (addr.addr.sa_family == AF_INET6) {
372     Dart_Handle scope_id_arg = Dart_GetNativeArgument(args, 3);
373     int64_t scope_id =
374         DartUtils::GetInt64ValueCheckRange(scope_id_arg, 0, 65535);
375     SocketAddress::SetAddrScope(&addr, scope_id);
376   }
377   intptr_t socket = Socket::CreateConnect(addr);
378   OSError error;
379   if (socket >= 0) {
380     Socket::SetSocketIdNativeField(Dart_GetNativeArgument(args, 0), socket,
381                                    Socket::kFinalizerNormal);
382     Dart_SetReturnValue(args, Dart_True());
383   } else {
384     Dart_SetReturnValue(args, DartUtils::NewDartOSError(&error));
385   }
386 }
```

The changes were successful; we can change the IP address of the proxy to our own.

Now the class_table.cc file:

```
ClassTable::Print() { OS::PrintErr("reFlutter");
 char pushArr[160000]="";
 Class& cls = Class::Handle();
 String& name = String::Handle();

 for (intptr_t i = 1; i < top_; i++) {
   if (!HasValidClassAt(i)) {
     continue;
   }
   cls = At(i);
   if (cls.ptr() != nullptr) {
     name = cls.Name();

    auto& funcs = Array::Handle(cls.functions());    if (funcs.Length()>1000) {    continue;    }» char classText[250000]="";  » Str
    strcat(classText,cls.ToCString());  »     Class& supcls = Class::Handle();    supcls = cls.SuperClass();  »    if (!supcls.IsNull
    strcat(classText," extends ");» »    strcat(classText,supname.ToCString());  »   }»  »     const auto& interfaces = Array::Handle
    Instance::Handle();»»     for (intptr_t in = 0;in < interfaces.Length(); in++) {»   interface^=interfaces.At(in);»  if(in==0){st
    {strcat(classText," , ");}» »   strcat(classText,interface.ToCString());»   }»  »     strcat(classText," {\n");»const auto& fiel
    Field::Handle();»   auto& fieldType = AbstractType::Handle(); »   String& fieldTypeName = String::Handle();»String& finame = Str
    Instance::Handle();»»     for (intptr_t f = 0; f < fields.Length(); f++)»   »   {    field ^= fields.At(f);»  finame = field.n
    fieldType.Name();»  strcat(classText,"  ");»»     strcat(classText,fieldTypeName.ToCString());»strcat(classText," ");» strcat(c
    if(field.is_static()){» »    »    instance2 ^= field.StaticValue();»  »    »    strcat(classText," = ");»  »    »    »    strcat(classTe
    ;\n");  } »   else {»     strcat(classText," = ");»   strcat(classText," nonstatic;\n");  }»}  »  for (intptr_t c = 0; c < funcs
    Function::Handle();    func = cls.FunctionFromIndex(c);  »    String& signature = String::Handle();    signature = func.Internal
    Code::Handle(func.CurrentCode());»    »    if(!func.IsLocalFunction()) {»»     strcat(classText," \n  ");»   strcat(classText,fun
    strcat(classText,signature.ToCString());»  »    »    strcat(classText," { \n\n              ");»     char append[70];»   sprintf(ap
    0x%016" PRIxPTR "\n",static_cast<uintptr_t>(codee.MonomorphicUncheckedEntryPoint()));»    strcat(classText,append);»»     strcat
     auto& parf = Function::Handle();» parf=func.parent_function();»  »    String& signParent = String::Handle();  »»     signPa
    strcat(classText," \n  ");» »  »    strcat(classText,parf.ToCString());»  strcat(classText," ");» strcat(classText,signParent.
    ");  »    char append[80];»   sprintf(append," Code Offset: _kDartIsolateSnapshotInstructions + 0x%016" PRIxPTR "\n",static_cast
     strcat(classText,append);»»     strcat(classText,"      \n      }\n");» »  }» }» »  »    strcat(classText," \n      }\n
    Library::Handle(cls.library());if (!libr.IsNull()) {  auto& owner_class = Class::Handle(); owner_class = libr.toplevel_class();
    Array::Handle(owner_class.functions());   char pushTmp[1000];   String& owner_name = String::Handle();   owner_name = libr.url()
    if (funcsTopLevel.Length()>0&&strstr(pushArr, pushTmp) == NULL) {  strcat(pushArr,pushTmp);   strcat(classText,"Library:"); strc
    for (intptr_t c = 0; c < funcsTopLevel.Length(); c++) {      auto& func = Function::Handle();    func = owner_class.FunctionFrom
     signature = func.InternalSignature();» auto& codee = Code::Handle(func.CurrentCode());»    if(!func.IsLocalFunction()) {»
    strcat(classText,func.ToCString());»strcat(classText," ");    strcat(classText,signature.ToCString());»  »    strcat(classText,"
    sprintf(append," Code Offset: _kDartIsolateSnapshotInstructions + 0x%016" PRIxPTR "\n",static_cast<uintptr_t>(codee.MonomorphicU
     strcat(classText,"      \n      }\n");»  »    } else {» »    auto& parf = Function::Handle();» parf=func.parent_function();
     signParent = parf.InternalSignature();»  »  »    strcat(classText," \n  ");»  »  »    strcat(classText,parf.ToCString()
    strcat(classText,signParent.ToCString());»  »    strcat(classText," { \n\n          "); »   char append[80];»   sprintf(appenc
    0x%016" PRIxPTR "\n",static_cast<uintptr_t>(codee.MonomorphicUncheckedEntryPoint()));»    strcat(classText,append);»»     strcat
    strcat(classText," \n      }\n\n");}}    struct stat entry_info;»    int exists = 0;    if (stat("/data/data/", &entry_info)==
     »  »   if(exists==1){»  »    pid_t pid = getpid();»»     char path[64] = { 0 };»  »    sprintf(path, "/proc/%d/cmdline",
```

This file contains an enumeration of classes, libraries, and functions, which we can modify as we want.

There is little time left before the compilation resumes, so it’s vital to modify the make_version.py file:

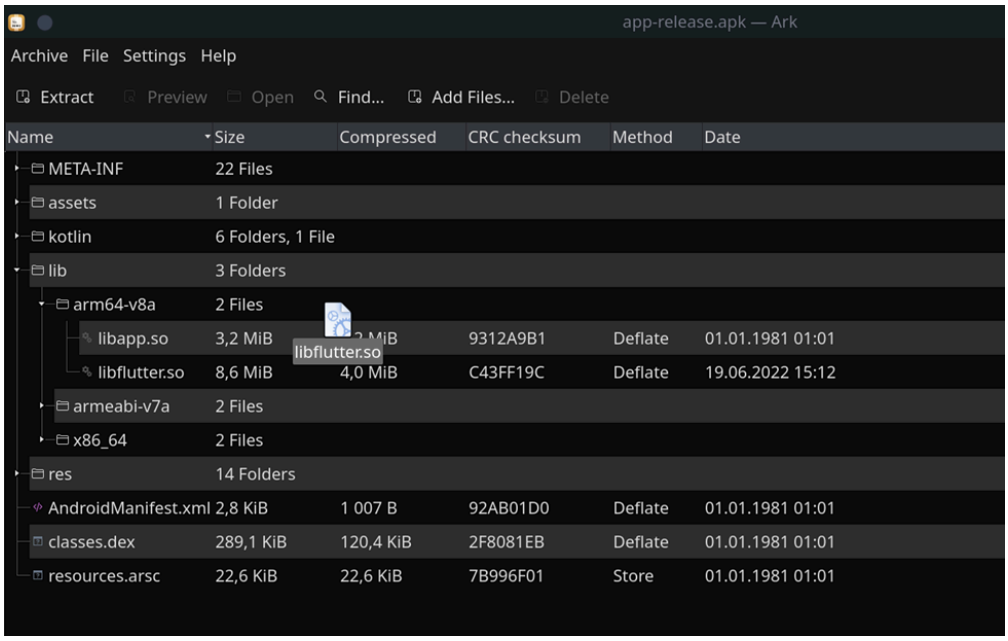
```
71
72      git_hash = None
73      # If we need SDK hash and git usage is not suppressed then try to get it.
74      if not no_sdk_hash and not no_git_hash:
75          git_hash = utils.GetShortGitHash()
76      if git_hash is None or len(git_hash) != 10:
77          git_hash = '0000000000'
78      version = version.replace('{{GIT_HASH}}', git_hash)
79
80      channel = utils.GetChannel()
81      version = version.replace('{{CHANNEL}}', channel)
82
83      version_time = None
84      if not no_git_hash:
85          version_time = utils.GetGitTimestamp()
86      if version_time == None:
87          version_time = 'Unknown timestamp'
88      version = version.replace('{{COMMIT_TIME}}', version_time.decode('utf-8'))
89
90      snapshot_hash = 'e8b7543ba0865c5bac45bf158bb3d4c1'
91      version = version.replace('{{SNAPSHOT_HASH}}', snapshot_hash)
92
93      return version
94
```

We need to modify the dummy snapshotHash value (5b97292b25f0a715613b7a28e0734f77) with the one extracted with reFlutter (e8b7543ba0865c5bac45bf158bb3d4c1).

Then the compilation resumes, and the compiled libflutter_arm64.so is saved as the output:

```
[3774/3786] STAMP obj/flutter/lib/snapshot/generate_snapshot_bin.stamp
[3775/3786] SOLINK ./libflutter.so
[3776/3786] ACTION //flutter/shell/platform/android:android_jar(//build/toolchain/android:clang_arm64)
[3777/3786] STAMP obj/flutter/shell/platform/android/android_jar.stamp
[3778/3786] ACTION //flutter/shell/platform/android:android_symbols(//build/toolchain/android:clang_arm64)
[3779/3786] STAMP obj/flutter/shell/platform/android/android_symbols.stamp
[3780/3786] ACTION //flutter/shell/platform/android:android(//build/toolchain/android:clang_arm64)
[3781/3786] STAMP obj/flutter/shell/platform/android/android.stamp
[3782/3786] STAMP obj/flutter/shell/platform/platform.stamp
[3783/3786] STAMP obj/flutter/shell/shell.stamp
[3784/3786] STAMP obj/flutter/sky/sky.stamp
[3785/3786] STAMP obj/flutter/flutter.stamp
[3786/3786] STAMP obj/default.stamp

'libflutter_arm64.so' -> '/t/libflutter_arm64.so'
c0de@c0de:~/AndroidStudioProjects/research$
```

Next, we rename the file to libflutter.so and replace it in the APK:



Great, the APK can be signed and run on the device.

This Docker image can be used to create not only a dev Engine but other engines too. This might be interesting if you want to develop your own patches or modify existing ones.

For example, to compile Engine 2.5.0, we just take SNAPSHOT_HASH and COMMIT from the table.

version	Engine_commit
....	....
2.5.0	f0826da7ef2d301eb8f4ead91aaf026aa2b52

We compile the stable version of the Engine on our PC:

```
sudo docker run -e WAIT=1000 -e x64=0 -e
arm=0 -e
HASH_PATCH=9cf77f4405212c45daf608e1cd64685
2 -e
```

```
COMMIT=f0826da7ef2d301eb8f4ead91aaf026aa2b
52881 --rm -iv${PWD}:/t ptswarm/reflutter
```

The output we get is libflutter_arm64.so.

Conclusion

Initially, my goal was to create a utility to help compile Flutter Engine, but then I decided to write a few patches of my own. Going forward, I hope the community will help to develop new ones, for example, for monitoring the file system. Unfortunately, my time is limited, but I will try to maintain the existing patches for new versions. There are a number of issues with the project. For example, the code offset is not always correct, and some functions are not extracted. Several additions to the project have appeared online, such as parser dump.dart, with the subsequent renaming of functions in IDA. Another patch is proposed for intercepting traffic without changing the socket but by rewriting the Environment functionality. Hopefully, reverse engineering of Flutter applications will improve in the future.

Мне очень понравилось исследовать Flutter и придумывать патчи. Я хотел бы поблагодарить сообщество за всю его поддержку и интерес к проекту, которые чрезвычайно помогли в разработке. И спасибо за чтение!

Автор

Филипп Никифоров
Эксперт по безопасности приложений

limpact_l

Безопасность мобильных приложений , Обратный инжиниринг

Предыдущий
Обнаружение доменов с помощью атаки на прозрачность сертификатов с использованием временной корреляции

Следующий
Возможности Jetty для взлома веб-приложений

Твиттер